ProspectIQ

Product Workflow & Low-Level Design (LLD)

Document type: Initial Product & Engineering Design
Status: Proposed architecture — for discussion and implementation planning
Version: 0.1
Scope: Seller onboarding, company research, evidence validation, prospect scoring, decision-maker intelligence, and outreach draft generation.

---

1. Executive Summary

ProspectIQ is an agentic B2B sales intelligence platform designed to help sales teams identify promising prospects, understand their business needs, discover relevant decision-makers, and prepare personalized outreach.

The system combines deterministic Python logic, LLM-based analysis, external data providers, structured validation, and evidence-backed reporting.

The core principle is:

Every important recommendation should be traceable to evidence, relevant to the seller's business, and explicit about uncertainty.

ProspectIQ should not simply summarize a company or generate generic sales messages. It should explain why a target company may be a suitable prospect for a particular seller and recommend an appropriate next action.

This document describes the intended product and architecture. It is not a claim that all described functionality is already implemented in the existing repository.

2. Problem Statement

B2B sales research is fragmented across company websites, news articles, hiring pages, business databases, professional profiles, and contact-discovery tools.

Salespeople must manually:

- Research a target company's products, market, growth, hiring, and expansion.
- Identify relevant buying signals and determine whether the company fits their offering.
- Find decision-makers and understand their responsibilities.

- Collect and verify supporting information.
- Develop a company-specific sales strategy.
- Write personalized messages for email, WhatsApp, and LinkedIn.

This process consumes time and can lead to irrelevant outreach, weak personalization, unsupported assumptions, and duplicated research.

2.1 Proposed solution

ProspectIQ will combine these activities into one workflow:

1. Understand the seller's company, products, services, and ideal customer profile (ICP).
2. Research a target company using multiple data sources.
3. Validate collected information and identify missing fields.
4. Retrieve missing information selectively.
5. Evaluate business signals and seller-specific prospect fit.
6. Generate an evidence-backed intelligence report.
7. Identify relevant decision-makers and build contact profiles.
8. Generate personalized outreach drafts for human review.

2.2 Product objectives

- Reduce repetitive manual prospect research.
- Improve relevance and quality of prospect prioritization.
- Avoid inventing facts or treating missing data as negative evidence.
- Preserve source attribution and explain scoring decisions.
- Minimize redundant API calls and LLM inference.
- Make research reproducible, testable, and maintainable.
- Keep outreach subject to explicit human approval.

3. Product Scope

3.1 In scope for the initial product

- User registration and authentication.
- Seller company onboarding.
- Product, service, and ICP extraction with user confirmation.
- Optional company documents and lead-requirement uploads.
- Target-company research.
- Web search, website extraction, news retrieval, and company-data integration.
- People search and decision-maker discovery.
- Structured evidence storage and schema validation.
- Missing-field detection and targeted retrieval.
- Deterministic scoring and LLM-generated explanations.
- Prospect intelligence reports.
- Contact profiles and personalized outreach drafts.

- Human review and approval before sending.

3.2 Out of scope for the first release

- Fully autonomous outreach without human approval.
- Complex multi-agent frameworks where a simple service or workflow is sufficient.
- Large-scale CRM synchronization.
- Multi-touch campaign optimization.
- Advanced predictive models trained on historical conversion data.
- Microservices and distributed orchestration before scale requires them.

4. End-to-End User Workflow

Step 1 — Registration and login

The user registers and authenticates. The system creates a seller workspace and associates subsequent company data, research runs, documents, and reports with that workspace.

Profile fields include name, email, LinkedIn URL, and company information. Authentication credentials and session management must be handled separately from the public profile.

Step 2 — Seller company setup

The user supplies their company name or website URL and may upload additional documents.

ProspectIQ collects information about:

- Company description, industry, and target market.
- Products and services.
- Value propositions and differentiators.
- Ideal customer profiles and target industries.
- Typical buyer roles, use cases, and known pain points.
- Additional sales requirements supplied by the user.

The system presents extracted information for review and correction before using it for prospect matching.

Step 3 — Target prospect submission

The user enters a target company name, website URL, or uploads a list of companies.

A research run is created for each target. The run is linked to the seller workspace, the applicable seller profile version, and the target company.

Step 4 — Research and evidence collection

The research orchestrator invokes the relevant tools:

- Web search for public company information.
- Website reader for official pages and relevant content.
- News retrieval for recent business developments.
- Company-data provider for structured company attributes.
- People-search provider, such as Prospeo, for relevant contacts.

Each returned item is normalized into a common evidence representation with source URL, retrieval time, source type, and relevant metadata.

Step 5 — Validation and targeted gap-filling

Pydantic validates the shape and constraints of the data. A separate completeness evaluator checks whether required research fields have sufficient evidence.

Examples of fields include hiring activity, expansion, funding, technology investment, company identity, and recent strategic changes.

When required fields are missing or insufficiently supported, the system selects a focused retrieval task for those fields only. It does not restart the entire research process unnecessarily.

The system stops when the required evidence criteria are met, the research budget is exhausted, or the remaining fields cannot be reliably obtained. Unresolved fields are explicitly marked as unknown.

Step 6 — Prospect scoring and intelligence report

Python computes deterministic weighted scores using validated inputs. The LLM interprets the collected evidence, explains relevant signals, and prepares a concise narrative.

The report includes:

- Company overview and current business context.
- Growth, hiring, expansion, and other relevant signals.
- Evidence-backed strengths, risks, and uncertainties.
- Seller-specific product fit.
- Prospect score, confidence, and recommendation.
- Why the prospect may need the seller's offering.
- Recommended approach and relevant talking points.

Step 7 — Decision-maker intelligence

The system identifies relevant people, their roles, and their potential relationship to the seller's offering.

Where available and permitted, it records contact details and profile evidence. It must distinguish verified facts from inferred responsibilities or suggested talking points.

Step 8 — Personalized outreach

ProspectIQ generates channel-specific drafts for relevant contacts:

- Email subject and body.
- WhatsApp message.
- LinkedIn connection note or message.

The user can review, edit, approve, or reject each draft. Sending is a separate action requiring explicit approval and channel-specific eligibility checks.

5. High-Level System Workflow

```
flowchart TD
    U[User] --> A[Authentication]
    A --> W[Seller Workspace]
    W --> S[Seller Company Setup]
    S --> D[Products, ICP and Documents]
    D --> P[Submit Target Company]
    P --> R[Create Research Run]

    R --> O[Research Orchestrator]
    O --> T[Search, Website, News and Company Data Tools]
    O --> C[People Search Tool]

    T --> E[Normalize Evidence]
    E --> V[Schema and Evidence Validation]
    V --> G{Required Evidence Complete?}

    G -- No --> M[Identify Missing or Weak Fields]
    M --> Q[Create Targeted Retrieval Tasks]
    Q --> O

    G -- Yes --> SC[Deterministic Prospect Scoring]
    SC --> L[LLM Report Synthesis]
    C --> CP[Contact Profiles]
    L --> REP[Prospect Intelligence Report]
    CP --> REP

    REP --> OUT[Personalized Outreach Drafts]
    OUT --> H[Human Review and Approval]
    H --> SEND[Approved Channel Connector]
    REP --> DB[(Persistent Storage)]
    CP --> DB
    OUT --> DB
    SEND --> LOG[Delivery and Audit Events]
    LOG --> DB
```

The diagram represents the intended workflow. Authentication, persistent storage, contact discovery, and the complete orchestration flow must be implemented and verified before this can be considered an end-to-end working product.

6. Proposed Logical Architecture

The initial application should use a modular architecture with clear boundaries. A modular monolith is sufficient for the first release: one deployable backend with independently testable modules.

6.1 Presentation and API layer

Responsibilities:

- Authentication and authorization.
- Seller setup and document upload endpoints.
- Prospect submission and research status.
- Report and contact retrieval.
- Outreach draft review and approval.

This layer should validate requests and delegate business operations to application services. It should not contain research logic or direct provider-specific code.

6.2 Application layer

Core use cases:

- `OnboardSeller`
- `BuildSellerProfile`
- `SubmitProspectResearch`
- `ExecuteResearchRun`
- `ValidateResearchCompleteness`
- `ScoreProspect`
- `GenerateProspectReport`
- `DiscoverDecisionMakers`
- `GenerateOutreachDrafts`
- `ApproveOutreachDraft`
- `SendApprovedOutreach`

Application services coordinate domain objects, repositories, and provider interfaces. They manage workflow transitions and failures without embedding external API details.

6.3 Domain layer

The domain layer owns business concepts and rules:

- Seller and workspace.
- Seller company, product, service, and ICP.
- Target prospect and business signals.
- Research run and evidence.
- Research field status and confidence.
- Prospect-fit assessment.
- Contact and contact profile.
- Outreach draft and approval state.

Domain logic should not depend on FastAPI, an LLM provider, Tavily, Prospeo, or a particular database.

6.4 Research and intelligence layer

Components:

- Research orchestrator.
- Research planner.
- Tool registry and execution policies.
- Evidence normalizer.
- Evidence evaluator.
- Completeness evaluator.
- Gap-driven retrieval planner.
- Scoring engine.
- Report synthesis service.

Python should control execution, limits, validation, scoring, and workflow state. The LLM should support interpretation, planning, and narrative generation within explicitly defined boundaries.

6.5 Infrastructure and integrations

Adapters implement interfaces for:

- Web search.
- Website reading.
- News retrieval.
- Structured company-data providers.
- People search and contact enrichment.
- LLM inference.
- Database persistence.
- Document/object storage.
- Email and messaging connectors.

Provider-specific response formats must not leak into the domain layer.

7. Low-Level Design — Core Components

7.1 Research orchestrator

The orchestrator coordinates a single research run. It should not implement every tool or analysis rule itself.

Proposed responsibilities:

1. Load the seller context and target company.
2. Create a research state with required fields and execution limits.
3. Request an initial research plan.
4. Execute approved tool calls.
5. Normalize and validate evidence.
6. Evaluate field completeness and evidence quality.
7. Schedule focused retrieval for unresolved fields.
8. Stop at completion or a configured limit.
9. Pass validated research to scoring and reporting.
10. Persist results and run status.

The orchestrator should be deterministic wherever possible. The model proposes actions; the application validates and authorizes them.

7.2 Tool contract

All research providers should implement a common interface.

Illustrative Python contract:

```
from typing import Protocol
from pydantic import BaseModel


class ResearchRequest(BaseModel):
    company_name: str
    company_domain: str | None = None
    requested_fields: list[str]
    query: str | None = None


class EvidenceItem(BaseModel):
    claim: str
    source_url: str
    source_type: str
    retrieved_at: str
    published_at: str | None = None
    confidence: float
    supporting_text: str


class ResearchTool(Protocol):
```

```
    name: str

    def execute(
        self,
        request: ResearchRequest,
    ) -> list[EvidenceItem]:
        ...
```

This is a design illustration, not a finalized implementation. The production schema should define timestamp types, confidence semantics, error types, source metadata, and provenance rules explicitly.

The orchestrator should depend on the interface, not on individual providers.

7.3 Research state

A research run should maintain explicit state rather than passing loosely structured dictionaries between agents.

Proposed fields:

- `research_run_id`
- `workspace_id`
- `prospect_id`
- `seller_profile_version`
- `status`
- `required_fields`
- `field_statuses`
- `evidence`
- `completed_tasks`
- `pending_tasks`
- `attempt_count`
- `budget_remaining`
- `started_at`
- `completed_at`
- `failure_reason`

Each required field should have a state such as:

- `unknown`
- `pending`
- `supported`
- `insufficient_evidence`
- `unavailable`

A field should not be marked supported merely because a value exists. Its evidence must meet the defined validation criteria.

7.4 Gap-driven retrieval

The gap-filling process should operate at the field level.

For example, if the initial research supports company identity, hiring, and expansion but lacks funding information, the system should schedule a funding-specific retrieval task.

```
flowchart TD
    A[Initial Evidence] --> B[Normalize and Validate]
    B --> C[Evaluate Required Fields]
    C --> D{Any Required Fields Unresolved?}

    D -- No --> E[Mark Research Complete]
    D -- Yes --> F[Rank Missing Fields]
    F --> G[Select Relevant Tool]
    G --> H[Execute Focused Retrieval]
    H --> I[Validate New Evidence]
    I --> J{Budget and Attempts Available?}
    J -- Yes --> C
    J -- No --> K[Mark Unresolved Fields]
    K --> L[Complete with Limitations]
```

Retrieval tasks should be deduplicated and bounded by maximum attempts, timeouts, and provider-specific limits. The system must not repeatedly call an LLM until it produces a desired answer.

7.5 Scoring engine

Scoring should separate two different questions:

A. Business signal score

How strongly does the evidence indicate growth, expansion, hiring, technology investment, or other business developments?

B. Seller-specific fit score

How closely does the target company's likely needs match the seller's products, services, ICP, and value proposition?

An initial weighted score could be:

$$
S_{\text{business}} =
\sum_{i=1}^{n} w_i s_i
$$

where each dimension score (s_i) is normalized to 0–100 and the weights sum to 1.

A separate seller-fit score should be calculated using explicitly defined matching criteria. A combined priority score can be introduced after the dimensions and their calibration are agreed upon.

Important rules:

- Score calculation and thresholds must be implemented in Python.
- LLM-generated scores must be validated and bounded.
- Missing evidence must not automatically be treated as a score of zero.
- Evidence freshness, source quality, and coverage should influence confidence.
- The report must explain the score and expose its major supporting evidence.
- Initial weights are hypotheses to validate, not proven predictors of conversion.

7.6 Report synthesis

The report service should receive validated evidence and deterministic scoring results.

The LLM may generate:

- Executive summary.
- Key signals and their business implications.
- Potential needs and use cases.
- Recommended sales approach.
- Evidence-backed risks and uncertainties.

It must not change the authoritative score or invent supporting facts.

Claims in the report should reference evidence identifiers so the user can inspect the underlying sources.

7.7 Contact discovery and outreach

Contact discovery should be a separate capability from company research.

The contact service should:

1. Search for people associated with the target company.
2. Match names, roles, and company identity.
3. Store available source evidence.
4. Enrich contact details through permitted provider integrations.
5. Build a profile that distinguishes facts from inferred information.
6. Rank contacts by relevance to the seller's offering.

The outreach service then combines the seller's profile, matched product, prospect report, and contact profile to generate a draft.

The approval and delivery pipeline must enforce authorization, contact eligibility, consent requirements where applicable, and do-not-contact rules immediately before sending.

8. Low-Level Design — SOLID Principles

S — Single Responsibility Principle

Each component should have one primary reason to change.

Examples:

- `WebSearchAdapter`: web-search provider integration.
- `EvidenceNormalizer`: standardize provider results.
- `CompletenessEvaluator`: determine which fields remain unresolved.
- `ProspectScoringService`: calculate deterministic scores.
- `ReportSynthesisService`: generate the narrative.
- `OutreachService`: prepare and manage drafts.

Avoid a single agent class that searches, scores, writes reports, persists records, and sends messages.

O — Open/Closed Principle

The system should allow new research providers to be added without rewriting orchestration.

For example, a new news provider should implement the research-tool contract and register through configuration. The orchestrator should not need provider-specific conditionals throughout its workflow.

L — Liskov Substitution Principle

Every adapter implementing a shared interface must honor its contract.

For example, all research tools should return normalized evidence or a documented provider error. A provider must not silently return fabricated records, change field semantics, or violate expected error behavior.

I — Interface Segregation Principle

Use small, capability-specific interfaces instead of one enormous provider interface.

For example:

- `WebSearchProvider`
- `WebsiteReader`
- `NewsProvider`
- `CompanyDataProvider`
- `PeopleSearchProvider`
- `LLMProvider`
- `EvidenceRepository`

A website reader should not be required to implement people search simply because both are research tools.

D — Dependency Inversion Principle

High-level business services should depend on abstractions, not concrete infrastructure.

For example, `ResearchOrchestrator` depends on `ResearchToolRegistry` and `EvidenceRepository` interfaces. Dependency injection supplies concrete Tavily, database, and other adapters.

The domain and application layers should remain testable without real external APIs.

9. Design Patterns

Use patterns only where they address an actual design problem.

Pattern	Application in ProspectIQ	Benefit
Strategy	Different research providers or scoring policies	Replace algorithms and providers without changing orchestration
Adapter	Tavily, Prospeo, LLM and messaging integrations	Normalize external APIs behind internal contracts
Factory / Registry	Construct and select enabled research tools	Centralize provider selection and configuration
Repository	Store and retrieve workspaces, evidence, research runs, and reports	Separate persistence from business logic
State	Manage research-run and outreach-draft lifecycle transitions	Make valid transitions explicit and testable
Template Method or Pipeline	Standardize normalization, validation, and processing stages	Keep shared processing consistent while allowing controlled variation
Dependency Injection	Supply provider implementations and repositories to services	Improve testability and reduce coupling
Policy / Specification	Express prospect-fit and evidence-acceptance rules	Make business rules explicit, composable, and independently testable

The orchestrator itself should initially be a normal application service with explicit control flow. Introduce a workflow engine only if retries, durable execution, parallel tasks, or recovery requirements justify it.

10. Component and Dependency Diagram

```
flowchart TB
    subgraph Presentation
        API[FastAPI Routes]
        AUTH[Authentication and Authorization]
    end

    subgraph Application
        ONB[Seller Onboarding Service]
        RS[Research Application Service]
        OS[Outreach Application Service]
    end

    subgraph Domain
        MODELS[Domain Models and Rules]
        SCORE[Prospect Scoring Policy]
        LIFECYCLE[Research and Draft State Rules]
    end

    subgraph Intelligence
        ORCH[Research Orchestrator]
        PLAN[Research Planner]
        NORM[Evidence Normalizer]
        VALID[Evidence and Completeness Evaluators]
        REPORT[Report Synthesis Service]
        CONTACT[Contact Intelligence Service]
    end

    subgraph Ports
        TOOLPORT[Research Tool Interfaces]
        LLMPORT[LLM Interface]
        REPOPORT[Repository Interfaces]
        MSGPORT[Messaging Interfaces]
    end

    subgraph Adapters
        SEARCH[Tavily Adapter]
        WEBSITE[Website Reader Adapter]
        NEWS[News Provider Adapter]
        COMPANY[Company Data Adapter]
        PEOPLE[Prospeo Adapter]
        LLM[MLX or Hosted LLM Adapter]
        DB[Database Repositories]
```

```
    DOCS[Document Storage Adapter]
    MSG[Channel Connectors]
  end

  API --> AUTH
  API --> ONB
  API --> RS
  API --> OS

  ONB --> MODELS
  RS --> ORCH
  RS --> SCORE
  RS --> REPORT
  ORCH --> PLAN
  ORCH --> NORM
  ORCH --> VALID
  ORCH --> TOOLPORT
  PLAN --> LLMPORT
  REPORT --> LLMPORT
  CONTACT --> TOOLPORT
  SCORE --> MODELS
  RS --> REPOPORT
  ONB --> REPOPORT
  OS --> REPOPORT
  OS --> MSGPORT

  TOOLPORT -. implemented by .-> SEARCH
  TOOLPORT -. implemented by .-> WEBSITE
  TOOLPORT -. implemented by .-> NEWS
  TOOLPORT -. implemented by .-> COMPANY
  TOOLPORT -. implemented by .-> PEOPLE
  LLMPORT -. implemented by .-> LLM
  REPOPORT -. implemented by .-> DB
  REPOPORT -. document storage .-> DOCS
  MSGPORT -. implemented by .-> MSG
```

The arrows represent logical dependencies, not a requirement to create a separate deployment for every component.

11. Core Data Model

The initial persistent model should support the following relationships:

```
erDiagram
  USER ||--o{ WORKSPACE : belongs_to
  WORKSPACE ||--|| SELLER_COMPANY : configures
  SELLER_COMPANY ||--o{ PRODUCT : offers
```

```
SELLER_COMPANY ||--o{ SELLER_DOCUMENT : provides
WORKSPACE ||--o{ PROSPECT : tracks
PROSPECT ||--o{ RESEARCH_RUN : researched_in
RESEARCH_RUN ||--o{ EVIDENCE : collects
PROSPECT ||--o{ BUSINESS_SIGNAL : exhibits
PROSPECT ||--o{ PROSPECT_ASSESSMENT : receives
PROSPECT ||--o{ CONTACT : has
CONTACT ||--o{ CONTACT_EVIDENCE : supported_by
CONTACT ||--o| CONTACT_PROFILE : described_by
PROSPECT_ASSESSMENT ||--o{ OUTREACH_DRAFT : informs
CONTACT ||--o{ OUTREACH_DRAFT : receives
OUTREACH_DRAFT ||--o{ OUTREACH_EVENT : records
```

Key entities

- **User / Workspace:** Identity, access control, and tenant boundary.
- **SellerCompany:** Company profile, offerings, and ICP.
- **Product:** Value proposition, target buyers, use cases, and differentiators.
- **SellerDocument:** Uploaded requirements or product documentation, with access-controlled storage.
- **Prospect:** Target company identity and seller-specific relationship.
- **ResearchRun:** Execution status, configuration version, budget, and timestamps.
- **Evidence:** Source URL, source type, retrieval time, extracted claim, and provenance.
- **BusinessSignal:** A normalized signal linked to supporting evidence.
- **ProspectAssessment:** Separate business and seller-fit scores, confidence, and explanation.
- **Contact / ContactProfile:** Decision-maker identity, role, evidence, and permitted contact channels.
- **OutreachDraft / OutreachEvent:** Message content, approval status, delivery attempts, and audit history.

Use stable identifiers and explicit workspace ownership. The exact SQL schema, indexes, retention rules, and migration strategy should be finalized during database design.

12. Failure Handling and Reliability

External research is inherently incomplete and unreliable. The system must treat provider failure as a normal operating condition.

Required behavior

- **Timeout:** Stop the individual request and record a retryable failure.
- **Rate limit:** Respect provider guidance and apply bounded backoff.
- **Invalid LLM output:** Validate against the expected schema and retry a limited number of times.
- **Missing evidence:** Record an unresolved field; do not fabricate a value.

- **Conflicting evidence:** Preserve both sources and mark the field as conflicting until a defined resolution policy applies.
- **Stale information:** Store retrieval and publication timestamps and apply field-specific freshness policies.
- **Research budget exhausted:** Produce a partial report with limitations rather than looping indefinitely.
- **Duplicate task:** Reuse valid results where appropriate and avoid redundant provider calls.
- **Delivery failure:** Record the failure and avoid marking an unsent message as sent.
- **Repeated request:** Use idempotency controls for research submission and sensitive delivery operations.

All research and outreach runs should have explicit status, error classification, timestamps, and observable logs.

13. Security and Data Governance

Security requirements must be part of the initial design, not added after the application grows.

- Enforce workspace-level authorization on every data-access operation.
- Store secrets in environment-specific secret management, never in source control.
- Treat uploaded files and scraped content as untrusted input.
- Restrict outbound fetches to safe public destinations and defend against SSRF, unsafe redirects, oversized responses, and malicious documents.
- Separate LLM instructions from retrieved content; retrieved pages must never be treated as trusted instructions.
- Validate tool arguments and restrict tool execution to explicitly permitted operations.
- Apply provider terms, applicable privacy laws, channel policies, and consent requirements.
- Avoid collecting unnecessary personal data and establish retention and deletion policies.
- Encrypt sensitive data in transit and at rest where appropriate.
- Audit access to seller documents, contact details, approvals, and delivery operations.

14. Testing Strategy

Testing should follow the architecture boundaries.

Unit tests

- Research field completeness and stopping conditions.
- Evidence normalization and source validation.
- Score calculations and recommendation thresholds.
- Seller-product-to-prospect matching.
- Research state transitions.
- Contact eligibility and do-not-contact rules.

- Draft approval and send-state transitions.

Integration tests

- Provider adapters using mocked HTTP responses.
- LLM structured-output parsing and bounded retries.
- Database repository operations.
- Research-to-report pipeline.
- Research-to-contact-to-outreach mapping.
- Delivery connector behavior in dry-run mode.

End-to-end tests

A complete seller workflow should verify:

1. Seller onboarding and company profile creation.
2. Submission of a target company.
3. Research and evidence collection.
4. Missing-field handling.
5. Prospect scoring and report generation.
6. Contact discovery or a documented no-results outcome.
7. Personalized draft generation.
8. Human approval and safe delivery handling.

Tests must not require paid provider calls by default. A smaller set of opt-in live-provider smoke tests can verify external integrations separately.

15. Implementation Roadmap

Phase 0 — Stabilize the foundation

- Audit the existing repository and current tests.
- Define domain schemas and service interfaces.
- Establish structured errors, logging, and configuration.
- Fix research-loop and evidence-evaluation weaknesses.
- Add deterministic tests for scoring and completeness.

Phase 1 — Reliable company research

- Integrate the website reader into the research workflow.
- Implement news and structured company-data provider adapters.
- Normalize and persist evidence.
- Implement required-field completeness checks.
- Add targeted gap-driven retrieval with strict budgets.
- Produce reproducible research reports.

Phase 2 — Seller context and prospect fit

- Implement seller onboarding.
- Build the seller company, product, and ICP profile.
- Add document upload and extraction.
- Map research reports to persistent prospect records.
- Introduce separate business-signal and seller-fit assessments.

Phase 3 — Decision-maker intelligence

- Integrate people search and contact enrichment.
- Validate company and role matching.
- Build evidence-backed contact profiles.
- Rank contacts by relevance to the seller's offering.

Phase 4 — Outreach integration

- Connect real research output to EXCON.
- Generate personalized channel-specific drafts.
- Enforce review, approval, consent, and do-not-contact policies.
- Implement delivery status tracking and audit events.

Phase 5 — User-facing application

- Deliver authenticated workspaces and research screens.
- Add research status, reports, contacts, and draft-review interfaces.
- Support saved reports, repeat research, and operational monitoring.
- Evaluate quality, cost, latency, and conversion-related outcomes.

The phases can overlap where dependencies permit, but seller-specific scoring depends on reliable seller context, and personalized outreach depends on real prospect and contact records.

16. Key Architectural Decisions

Decision	Initial choice	Reason
Deployment architecture	Modular monolith	Simple deployment with clean internal boundaries
Orchestration	Explicit Python workflow	Easier to test, debug, and constrain
Data validation	Pydantic schemas plus domain rules	Separate structure validation from evidence completeness
LLM responsibility	Planning assistance and synthesis	Avoid delegating deterministic business rules to a model
Scoring	Python-based weighted scoring	Reproducible and explainable calculations

External integrations	Adapter interfaces	Providers can be replaced independently
Persistence	Repository interfaces backed by a database	Durable records and future implementation flexibility
Research recovery	Bounded retries and targeted retrieval	Control cost and prevent infinite loops
Outreach	Human approval before sending	Reduce risk and retain user control
Framework adoption	Only when justified	Avoid premature complexity

17. Open Questions Before Implementation

The following decisions require validation before finalizing the design:

1. Which database and document-storage system will be used?
2. Which company-data and news providers are available, licensed, and cost-effective?
3. What is the precise contract and coverage of the current Prospeo integration?
4. Which research fields are mandatory versus optional?
5. How should evidence freshness, conflicts, and confidence be calculated?
6. What scoring weights and thresholds should be used initially, and how will they be evaluated?
7. How should seller products and ICPs be represented when a company has multiple offerings?
8. Which user-facing authentication and authorization model is required?
9. Which channels can be safely supported for actual sending, given their API and consent requirements?
10. What latency and cost budgets should apply to a single research run?

These questions should be resolved incrementally, based on the existing code, provider capabilities, and initial product requirements.

18. Definition of Done for the Initial Research Engine

The initial research engine should be considered ready for integration only when:

- Research requests produce structured, validated results.
- Every material signal can be traced to supporting evidence.
- Missing fields are explicitly identified and selectively researched.
- Retries, execution budgets, and termination conditions are enforced.
- Scoring is deterministic for the same validated input and policy version.
- The report distinguishes evidence, interpretation, and uncertainty.
- Provider failures produce controlled outcomes rather than fabricated results.

- Research runs and evidence can be persisted and retrieved.
- Automated tests cover successful, partial, and failed research scenarios.
- The pipeline can run without depending on mock contacts or mock seller profiles.

Conclusion

ProspectIQ should be built as an evidence-driven sales intelligence system with explicit workflow control, well-defined domain models, replaceable integrations, deterministic scoring, and carefully bounded LLM responsibilities.

The most important engineering priority is to make the research pipeline trustworthy before expanding the application surface. Once research, evidence, and seller-specific fit are reliable, contact intelligence and personalized outreach can be integrated without coupling every part of the product to one large agent.

This document establishes the proposed baseline. It should be revised as implementation reveals provider constraints, testing results, and product requirements. Existing code should be mapped against this design before individual modules are rewritten or new infrastructure is introduced.